Simulation and Bootstrap

Statistical Computing, STA3005

Mar 30, 2026

Last chapter: Modeling

- Fitting models is critical to both statistical inference and prediction
- Exploratory data analysis is a very good first step and gives you a sense of what you're dealing with before you start modeling
- Linear regression is the most basic modeling tool of all, and one of the most ubiquitous
- `lm()` allows you to fit a linear model by specifying a formula, in terms of column names of a given data frame
- Utility functions `coef()`, `fitted()`, `residuals()`, `summary()`, `plot()`, `predict()` are very handy and should be used over manual access tricks
- Logistic regression is the natural extension of linear regression to binary data; use `glm()` with `family="binomial"` and all the same utility functions
- Generalized additive models add a level of flexibility in that they allow the predictors to have nonlinear effects; use `gam()` and utility functions

Part I: *Simulation basics*

Why simulate?

R gives us unique access to great simulation tools (unique compared to other languages). Why simulate? Welcome to the 21st century! Two reasons:

- Often, simulations can be easier than hand calculations
- Often, simulations can be made more realistic than hand calculations

Sampling from a given vector

To sample from a given vector, use `sample()`

```
sample(x=letters, size=10) # Without replacement, the default
```

```
##  [1] "p" "v" "e" "l" "o" "i" "x" "f" "z" "d"
```

```
sample(x=c(0,1), size=10, replace=TRUE) # With replacement
```

```
##  [1] 1 0 1 1 1 0 1 1 1 1
```

```
sample(x=10) # Arguments set as x=1:10, size=10, replace=FALSE
```

```
##  [1]  8  4  9  5 10  3  6  7  1  2
```

```
# Sample with unequal probabilities
pmf <- dbinom(0:3, 3, 0.5)
sample(0:3, size = 10, replace = TRUE, prob = pmf) # Sample from Binomial(3,0.5)
```

```
##  [1] 3 1 1 0 3 2 2 1 2 1
```

Random number generation

To sample from a normal distribution, we have the utility functions:

- `rnorm()` : generate normal random variables
- `pnorm()` : normal distribution function, $\Phi(x) = P(Z \leq x)$
- `dnorm()` : normal density function, $\phi(x) = \Phi'(x)$
- `qnorm()` : normal quantile function, $q(y) = \Phi^{-1}(y)$, i.e., $\Phi(q(y)) = y$

Replace “norm” with the name of another distribution, all the same functions apply. E.g., “t”, “exp”, “gamma”, “chisq”, “binom”, “pois”, etc.

Random number examples

Standard normal random variables (mean 0 and variance 1)

```
n = 100
z = rnorm(n, mean=0, sd=1) # These are the defaults for mean, sd
mean(z)  # Check: sample mean is approximately 0
```

```
## [1] 0.143354
```

```
var(z)   # Check: sample variance is approximately 1
```

```
## [1] 1.038873
```

Estimated distribution function

To compute empirical cumulative distribution function (ECDF)—the standard estimator of the cumulative distribution function (CDF)—use `ecdf()`

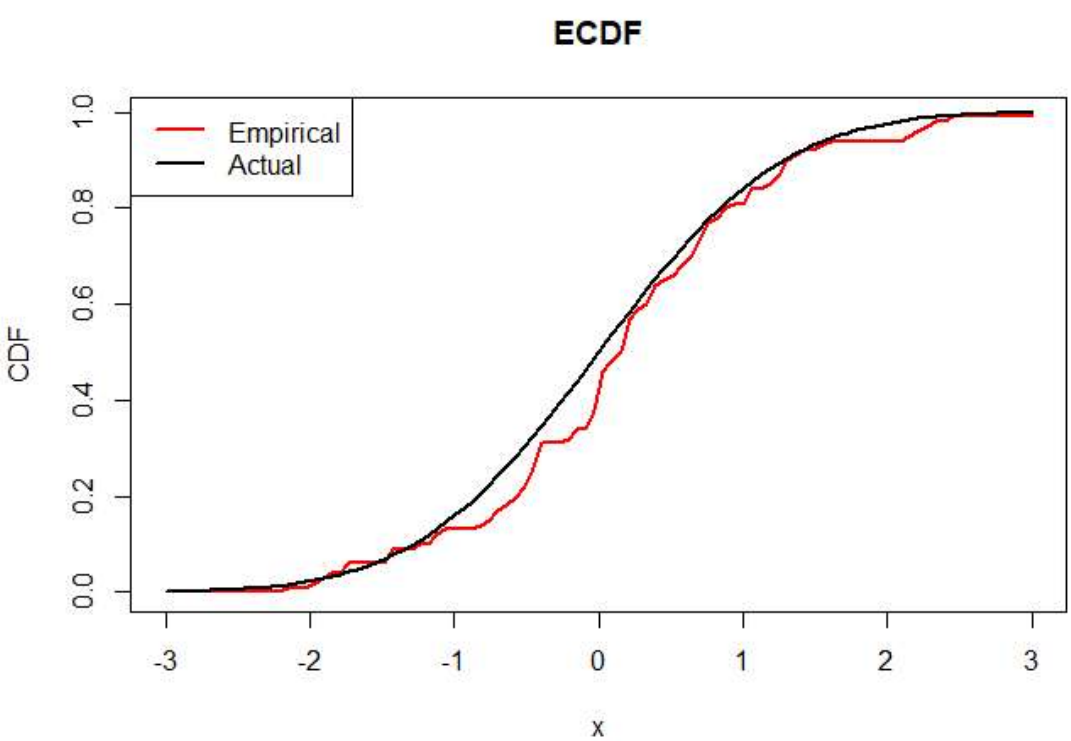
```
x = seq(-3, 3, length=100)
ecdf.fun = ecdf(z) # Create the ECDF
class(ecdf.fun) # It's a function!
```

```
## [1] "ecdf"     "stepfun"  "function"
```

```
ecdf.fun(0)
```

```
## [1] 0.4
```

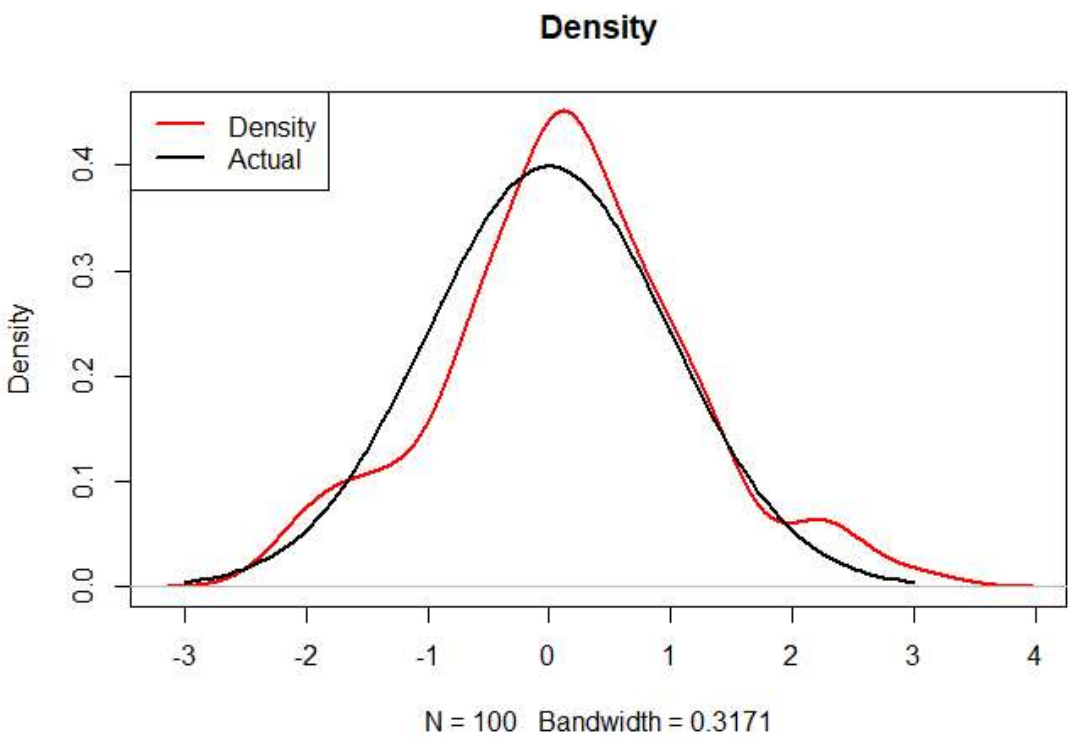
```
# We can plot it
plot(x, ecdf.fun(x), lwd=2, col="red", type="l", ylab="CDF", main="ECDF")
lines(x, pnorm(x), lwd=2)
legend("topleft", legend=c("Empirical", "Actual"), lwd=2,
       col=c("red","black"))
```



Estimated density function

We can also compute empirical density function by `density()`

```
# We can plot it
plot(density(z), col = "red", lwd = 2, main="Density")
lines(x, dnorm(x), lwd=2)
legend("topleft", legend=c("Density", "Actual"), lwd=2,
       col=c("red","black"))
```



Part II: *Pseudorandomness and seeds*

Same function call, different results

Not surprisingly, we get different draws each time we call `rnorm()`

```
mean(rnorm(n))
```

```
## [1] 0.01690829
```

```
mean(rnorm(n))
```

```
## [1] 0.1436944
```

```
mean(rnorm(n))
```

```
## [1] -0.02899128
```

```
mean(rnorm(n))
```

```
## [1] 0.05683938
```

Is it really random?

Random numbers generated in R (in any language) are not "truly" random; they are what we call **pseudorandom**

- These are numbers generated by computer algorithms that very closely mimick "truly" random numbers
- The study of such algorithms is an interesting research area in its own right!
- The default algorithm in R (and in nearly all software languages) is called the "Mersenne Twister", which a kind of congruetial generators.

Mersenne-Twister

Consider an initial number X_0 and three big integers A, B and m. The next random integer is generated by

$$X_{n+1} \equiv (AX_n + B) \mod m$$

Then, $U_i = \frac{X_i}{m} \in [0, 1), i = 1, 2, \cdots$ can be regarded as a sequence of random number generated from 0-1 uniform distribution.

- For example, $m = 2^{32}, A = 1,664,525, B = 1,013,904,223$ for a huge period
- Always repeat after sufficient large number of iterations
- Type `?Random` in your R console to read more about this (and to read how to change the algorithm used for pseudorandom number generation, which you should never really have to do, by the way)

Setting the random seed

All pseudorandom number generators depend on what is called a **seed** value

- This puts the random number generator in a well-defined "state", so that the numbers it generates, from then on, will be reproducible
- The seed is just an integer, and can be set with `set.seed()`

- The reason we set it: so that when someone else runs our simulation code, they can see the same—albeit, still random—results that we do

Seed examples

```
# Getting the same 5 random normals over and over
set.seed(0); rnorm(5)
```

```
## [1]  1.2629543 -0.3262334  1.3297993  1.2724293  0.4146414
```

```
set.seed(0); rnorm(5)
```

```
## [1]  1.2629543 -0.3262334  1.3297993  1.2724293  0.4146414
```

```
set.seed(0); rnorm(5)
```

```
## [1]  1.2629543 -0.3262334  1.3297993  1.2724293  0.4146414
```

```
# Different seeds, different numbers
set.seed(1); rnorm(5)
```

```
## [1] -0.6264538  0.1836433 -0.8356286  1.5952808  0.3295078
```

```
set.seed(2); rnorm(5)
```

```
## [1] -0.89691455  0.18484918  1.58784533 -1.13037567 -0.08025176
```

```
set.seed(3); rnorm(5)
```

```
## [1] -0.9619334 -0.2925257  0.2587882 -1.1521319  0.1957828
```

```
# Each time the seed is set, the same sequence follows (indefinitely)
set.seed(0); rnorm(3); rnorm(2); rnorm(1)
```

```
## [1]  1.2629543 -0.3262334  1.3297993
```

```
## [1] 1.2724293 0.4146414
```

```
## [1] -1.53995
```

```
set.seed(0); rnorm(1); rnorm(2); rnorm(3)
```

```
## [1] 1.262954
```

```
## [1] -0.3262334  1.3297993
```

```
## [1]  1.2724293  0.4146414 -1.5399500
```

```
set.seed(0); rnorm(6)
```

```
## [1]  1.2629543 -0.3262334  1.3297993  1.2724293  0.4146414 -1.5399500
```

Part III: *Simulation methods*

Generate discrete random variables by inverse method

Suppose we have a sequence of 0-1 uniform random number $U_i, i = 1, 2, \cdots, N$. We want to generate X_i i.i.d. from a discrete distribution with a known probability distribution

$$Pr(X_i \leq x) = p_x, x = 0, 1, 2, \cdots$$

Let $X_i = x$, if $U_i \leq p_x$ and $U > p_{x-1}$. Then, $X_i \sim p_x$.

Example

Draw from Binomial(3,0.5)

```
# Inverse method
U <- runif(1000)
X_inv <- rep(NA, 1000)
cdf <- pbinom(0:3, 3, 0.5)

for(i in 1:1000){
  X_inv[i] <- sum(U[i] > cdf)
```

```
}

table(X_inv)/1000
```

```
## X_inv
##     0     1     2     3
## 0.119 0.399 0.353 0.129
```

```
pmf <- dbinom(0:3, 3, 0.5)
pmf
```

```
## [1] 0.125 0.375 0.375 0.125
```

Generate continuous random variables by inverse method

Instead, we want to generate X_i i.i.d. from a continuous distribution with a known probability distribution

$$Pr(X_i \leq x) = F(x)$$

Let $X_i = F^{-1}(U_i)$, where F^{-1} is the inverse function of F. Then, $X_i \sim F(x)$.

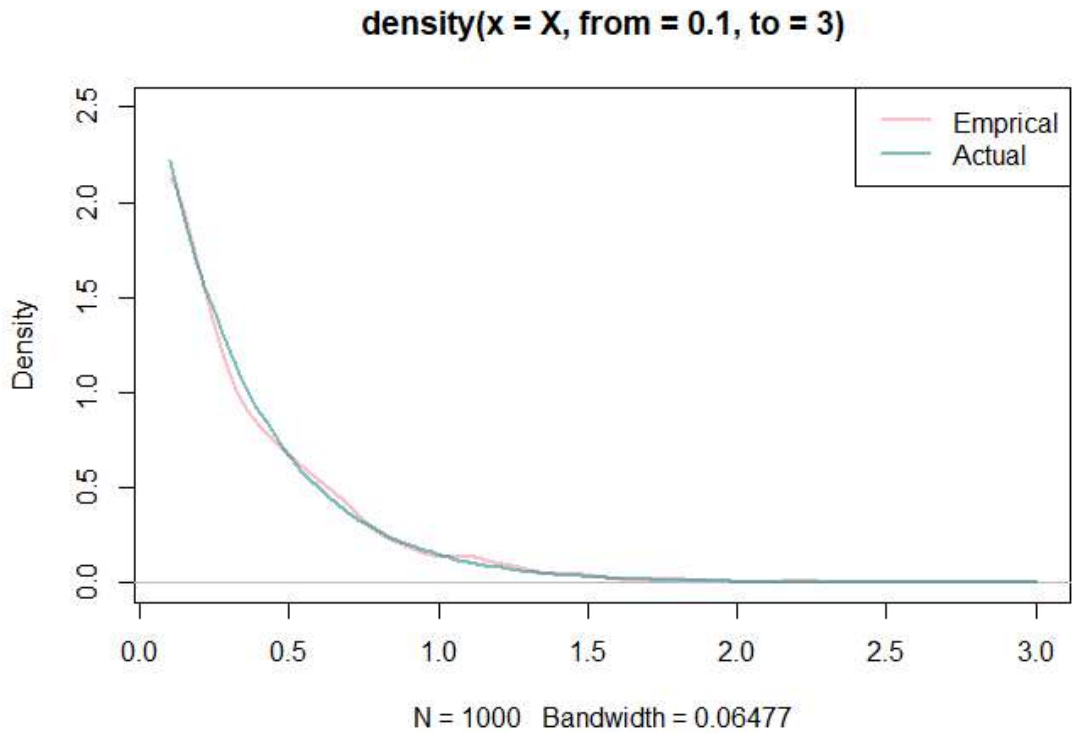
Example

Draw from Exp(3)

$F(x) = 1 - \exp(-3x) \Rightarrow x = F^{-1}(u) = -\frac{\log(1-u)}{3}$

```
# Inverse method
n <- 1000
U <- runif(n)
X <- - log(1-U)/3
plot(density(X, from  = 0.1, to = 3),  col = "pink", ylim = c(0,2.5), lwd = 2)

x_seq = seq(0.1, 3, length=100)
lines(x_seq, dexp(x_seq,3), lwd=2, col = rgb(0,0.5,0.5,0.5))
legend("topright", legend=c("Emprical", "Actual"), lwd=2,
       col=c("pink",rgb(0,0.5,0.5,0.5)))
```



Generate random variables by the acceptance-rejection method

The Acceptance-Rejection Method

To sample a random variable X with density f:

1. Find a random variable Y with density g such that we can directly generate random variables from g and $\frac{f(t)}{g(t)} \leq c$, for all t such that $f(t) > 0$.

2. For each random variable X:

    a. Generate a random y from the distribution with density g.

    b. Generate a random u from the $\text{Uniform}(0, 1)$ distribution.

    c. If $u < \frac{f(y)}{cg(y)}$, accept y and deliver $X = y$; otherwise reject y and repeat from Step 2a.

Why it works?

Here, we prove for discrete random variables

$$P(X=x) = P(Y=x|\text{accept}) = \frac{P(Y=x,\text{accept})}{P(\text{accept})} = \frac{P(\text{accept}|Y=x)P(Y=x)}{\sum_y P(\text{accept} \mid Y=y)P(Y=y)}$$

The acceptance rate given $Y=x$ is

$$P(\text{accept} \mid Y=x) = P\left(U < \frac{f(Y)}{cg(Y)}\Big|Y=x\right) = \frac{f(x)}{cg(x)}.$$

The total probability of acceptance in the denominator is therefore

$$\sum_y P(\text{accept} \mid Y=y)P(Y=y) = \sum_y \frac{f(y)}{cg(y)}g(y) = \frac{1}{c}.$$

We finally obtain

$$P(X=x) = \frac{f(x)}{cg(x)} \cdot g(x) \cdot c = f(x)$$

The proof for continous cases is similar.

Example

Draw from Beta(2, 2) distribution by the acceptance-rejection method

- Target distribution: $\text{Beta}(\alpha=2, \beta=2)$
- Density function: $f(x) = 6x(1-x)$, for $0 < x < 1$
- Proposal density: $g(x) = \text{Uniform}(0,1)$
- Constant: $c = 6$ (since $f(x)/g(x) \leq 6$ for all x)

Algorithm:

1. Generate $y \sim \text{Uniform}(0,1)$
2. Generate $u \sim \text{Uniform}(0,1)$
3. Accept y if $u \leq \frac{f(y)}{c \cdot g(y)} = \frac{6y(1-y)}{6 \cdot 1} = y(1-y)$

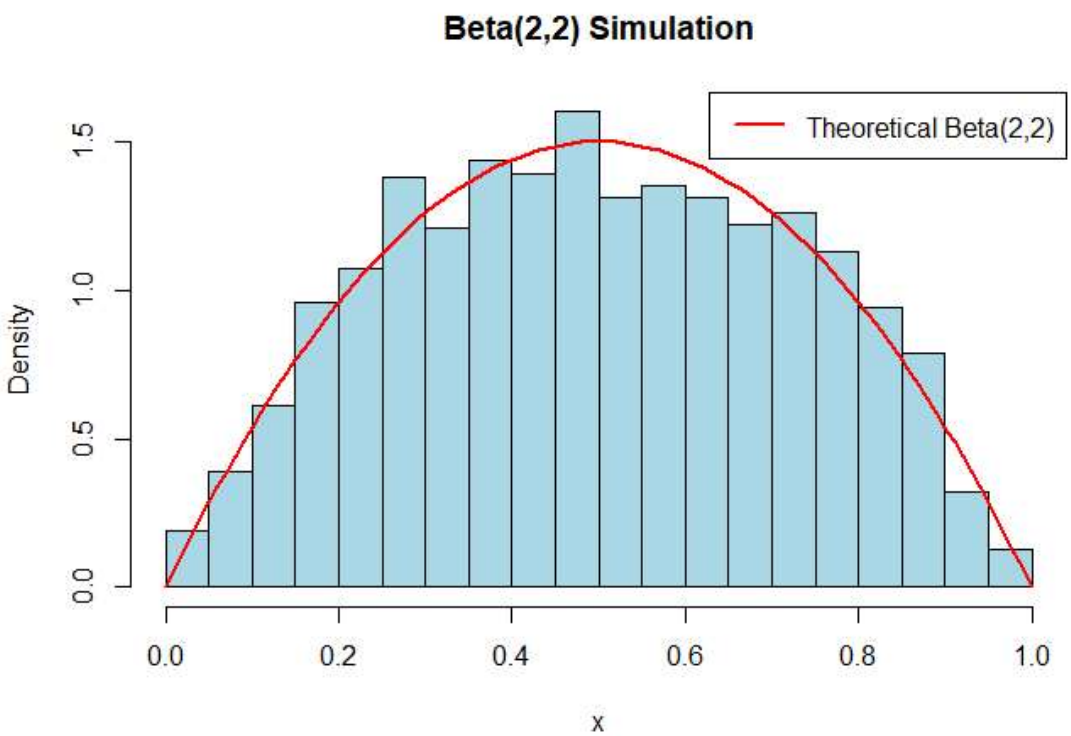
Simulation Setup:

```
n <- 2000        # Target sample size
k <- 0           # Accepted samples counter
j <- 0           # Total iterations counter
y <- numeric(n)  # Storage for accepted values

set.seed(123)
while (k < n) {
u <- runif(1)
j <- j + 1
x <- runif(1) #random variate from g
if (x * (1-x) > u) {
```

```
#we accept x
k <- k + 1
y[k] <- x
}
}

hist(y, probability = TRUE, main = "Beta(2,2) Simulation",
     xlab = "x", col = "lightblue", breaks=30)
curve(dbeta(x, 2, 2), add = TRUE, col = "red", lwd = 2)
legend("topright", legend = "Theoretical Beta(2,2)",
       col = "red", lwd = 2)
```




```
# Actaul acceptance rate
n/j
```

```
## [1] 0.166251
```

Part IV: *Iteration and simulation*

Drug effect model

Let's start with a motivating example: suppose we had a model for the way a drug affecting certain patients. All patients will undergo chemotherapy no matter whether they take the drug.

- Those who didn't take the drug experience a reduction in tumor size of percentage

$$X_{\text{no drug}} \sim 10 \cdot \text{Exp}(\text{mean} = R), \quad R \sim \text{Unif}(0, 1)$$

- Those who took the drug experience a reduction in tumor size of percentage

$$X_{\text{drug}} \sim 10 \cdot \text{Exp}(\text{mean} = 2)$$

(Here Exp denotes the exponential distribution, and Unif does the uniform distribution)

What would you do?

What would you do if you had such a model, and your scientist collaborators asked you:

How many patients would we need to have in each group (drug, no drug), in order to reliably see that the average reduction in tumor size is significantly different between two groups for a two-sample t-test?

- Answer used to be: get out your pen and paper, make some approximations
- Answer is now: simulate from the model, no approximations required!

So, let's simulate!

```
# Simulate, supposing 60 subjects in each group
set.seed(0)
n = 60
mu.drug = 2
mu.nodrug = runif(n, min=0, max=1)
x.drug = 10*rexp(n, rate=1/mu.drug)
x.nodrug = 10*rexp(n, rate=1/mu.nodrug)
```

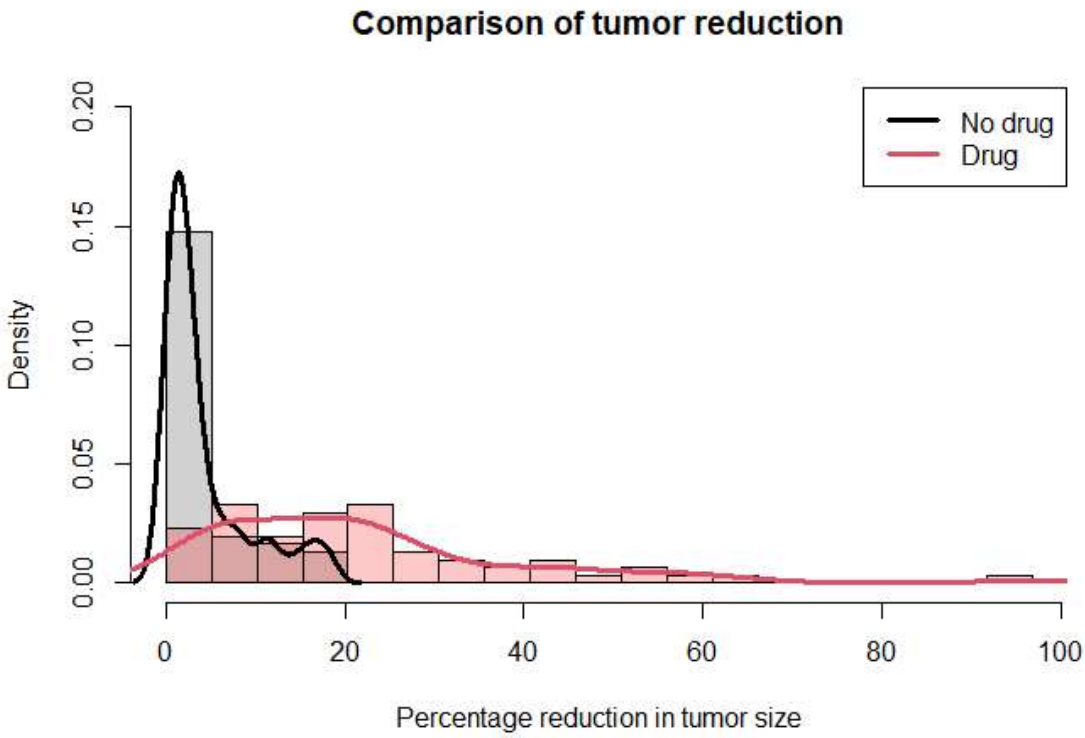
```
# Find the range of all the measurements together, and define breaks
x.range = range(c(x.nodrug,x.drug))
breaks = seq(min(x.range),max(x.range),length=20)

# Produce hist of the non drug measurements, then drug measurements on top
hist(x.nodrug, breaks=breaks, probability=TRUE, xlim=x.range, ylim = c(0,0.2),
     col="lightgray", xlab="Percentage reduction in tumor size",
     main="Comparison of tumor reduction")

# Plot a histogram of the drug measurements, on top
hist(x.drug, breaks=breaks, probability=TRUE, col=rgb(1,0,0,0.2), add=TRUE)

# Draw estimated densities on top, for each dist
lines(density(x.nodrug), lwd=3, col=1)
```

```
lines(density(x.drug), lwd=3, col=2)
legend("topright", legend=c("No drug","Drug"), lty=1, lwd=3, col=1:2)
```




```
# Two sample t-test
test <- t.test(x.nodrug, x.drug)
test
```

```
##
##  Welch Two Sample t-test
##
## data:  x.nodrug and x.drug
## t = -7.401, df = 67.675, p-value = 2.743e-10
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  -22.69881 -13.05739
## sample estimates:
## mean of x mean of y
##  4.255374 22.133476
```

Repetition and reproducibility

- One single simulation is not always trustworthy (depends on the situation, of course)

- In general, simulations should be repeated and aggregate results reported—requires iteration!
- To make random number draws reproducible, we must set the seed with `set.seed()`
- More than this, to make simulation results reproducible, we need to follow **good programming practices**
- Gold standard: any time you show a simulation result (a figure, a table, etc.), you have code that can be run (by anyone) to produce exactly the same result

```
drug.sim = function(n.sample, mu.drug=2, mu.nodrug.range=c(0,1)) {
  mu.nodrug <- runif(n.sample, mu.nodrug.range[1], mu.nodrug.range[2])
  x.drug = 10*rexp(n.sample, rate=1/mu.drug)
  x.nodrug = 10*rexp(n.sample, rate=1/mu.nodrug)
  drug.test <- t.test(x.drug, x.nodrug)
  return(drug.test$p.value)
}
```

Iteration and simulation (and functions): good friends

- Writing a **function** to complete a single run of your simulation is often very helpful
- This allows the simulation itself to be intricate (e.g., intricate steps, several simulation parameters), but makes running the simulation simple
- Then you can use **iteration** to run your simulation over and over again
- Good design practice: write another function for this last part (running your simulation many times)

Code sketch

Consider the code below for a generic simulation. Think about how you would frame this for the drug effect example, which you'll revisit in the assignment

```
# Function to do one simulation run
one.sim = function(param1, param2=value2, param3=value3) {
  # Possibly intricate simulation code goes here
}

# Function to do repeated simulation runs
rep.sim = function(nreps, param1, param2=value2, param3=value3, seed=NULL) {
  # Set the seed, if we need to
  if(!is.null(seed)) set.seed(seed)

  # Run the simulation over and over
  sim.objs = vector(length=nreps, mode="list")
  for (r in 1:nreps) {
    sim.objs[r] = one.sim(param1, param2, param3)
  }
```

```
  # Aggregate the results somehow, and then return something
}
```

Saving results

Sometimes simulations take a long time to run, and we want to save intermediate or final output, for quick reference later

There two different ways of saving things from R (there are more than two, but here are two useful ones):

- `saveRDS()` : allows us to save single R objects (like a vector, matrix, list, etc.), in (say) .rds format. E.g.,

  ```
  saveRDS(my.mat, file="my.matrix.rds")
  ```

- `save()` : allows us to save any number of R objects in (say) .rdata format. E.g.,

  ```
  save(mat.x, mat.y, list.z, file="my.objects.rdata")
  ```

- `save.image()` : allows use to save the entire global enviroment (workspace) in .rdata format. E.g.,

  ```
  save.image(file="my.wk.space.rdata")
  ```

Note: there is a big difference between how these two treat variable names

Loading results

Corresponding to the two different ways of saving, we have two ways of loading things into R:

- `readRDS()` : allows us to load an object that has been saved by `savedRDS()` , and **assign a new variable name**. E.g.,

  ```
  my.new.mat = readRDS("my.matrix.rds")
  ```

- `load()` : allows us to load all objects that have been saved through `save()` , **according to their original variables names**. E.g.,

  ```
  load("my.objects.rdata")
  ```

Note: you can also load the entire workspace by `load()`

Part V: *Bootstrap*

Bootstrap methods are a class of nonparametric Monte Carlo methods that estimate the distribution of a population by **resampling**

- Treat an observed sample as a finite "population"
- Generate random bootstrap samples from the finite "population"
- Approximate the unknown distribution of an estimator of interest by the empirical distribution of bootstrap samples

Motivation

To make inference on θ of a distribution with CDF F. The parameter $\theta = t(F)$ is defined as a function of F, say

- k-th moment: $\mu_k(F) = \int x^k \, dF(x)$,
- k-th central moment: $\omega_k(F) = \int (x - \mu_1(F))^k \, dF(x)$
- p-th quantile: $Q_p(F) = \inf\{x : F(x) \geq p\}$

Based on a sample $X_1, \ldots, X_n$ drawn from the distribution F, we are interested in **the sampling properties of an estimator** $\hat{\theta} = T(X_1, \ldots, X_n)$ for θ, like its variance, bias and distribution.

Example

If $X_1, \ldots, X_n$ are drawn from a normal distribution $N(\mu, 1)$, we might use

- the sample mean ($\bar{X}$)
- the median

to estimate μ.

However, we only one sample!

Intuition

Use the empirical distribution function (EDF) built by the sample $x_1, \ldots, x_n$ to approximate the true distribution F.

$$F_n \to F \; a.s., \text{ as } n \to \infty$$

In particular, the empirical cumulative distribution function (ECDF) $F_n(x)$ is defined as:

$$F_n(x) = \frac{1}{n}\sum_{i=1}^{n} I\{X_i \leq x\}$$

where $I\{X_i \leq x\}$ is the indicator function.

How to conduct

For each bootstrap replicate, indexed by $b = 1, \ldots, B$:

1. Generate sample $x^{*(b)} = \{x_1^*, \ldots, x_n^*\}$ by **sampling with replacement** from the observed sample $x_1, \ldots, x_n$.
2. Compute the bth replicate $\hat{\theta}^{(b)} = T(x^{*(b)})$ from the bth bootstrap sample.
3. The bootstrap estimate of F_θ is the empirical distribution of the replicates $\hat{\theta}^{(1)}, \ldots, \hat{\theta}^{(B)}$.

Example

We observe the sample $x = \{2, 2, 1, 1, 5, 4, 4, 3, 1, 2\}$ generated from a $Pois(2)$ distribution.

Bootstrap resamples x from 1, 2, 3, 4, or 5 with probabilities 0.3, 0.3, 0.1, 0.2, and 0.1, respectively.

$$F_n(x) = \begin{cases} 0, & x < 1; \\ 0.3, & 1 \leq x < 2; \\ 0.6, & 2 \leq x < 3; \\ 0.7, & 3 \leq x < 4; \\ 0.9, & 4 \leq x < 5; \\ 1, & x \geq 5. \end{cases}$$

- Note that if F_n is not close to F, then the distribution F_B^* of the replicates will not be close to F.
- Resampling from x a large number of replicates produces a good estimate of F_n but not necessarily a good estimate of F

$$F_B^* \to F_n \ a.s., \text{ as } B \to \infty$$

- Regardless of how many replicates are drawn, the bootstrap samples will never include 0.

Bootstrap Approximations

In bootstrap, there are two approximations:

- The ecdf F_n is an approximation to the cdf F.
- The ecdf F_B^* of the bootstrap replicates is an approximation to the ecdf F_n.

Resampling from the sample x is equivalent to generating random samples from the distribution $F_n(x)$. The two approximations can be represented by:

$$F \to x_i, i = 1, 2, \ldots, n \to F_n$$

$$F_n \to x^{*(b)}, b = 1, 2, \ldots, B \to F_B^*.$$

Bootstrap estimation of a standard error

The bootstrap estimate of standard error of an estimator $\hat{\theta}$ is the sample standard deviation of the bootstrap replicates $\hat{\theta}^{(1)}, \ldots, \hat{\theta}^{(B)}$.

$$\widehat{se}\left(\hat{\theta}^*\right) = \sqrt{\frac{1}{B-1}\sum_{b=1}^{B}\left(\hat{\theta}^{(b)} - \overline{\hat{\theta}^*}\right)^2},$$

where $\overline{\hat{\theta}^*} = \frac{1}{B}\sum_{b=1}^{B}\hat{\theta}^{(b)}$.

- $B = 50$ is usually large enough, rarely is $B > 200$ necessary
- Much larger B will be needed for confidence interval estimation

Example

The law school data set `law` in the `bootstrap` package contains LSAT (average score on law school admission test score) and GPA (average undergraduate grade point average) for 15 law schools.

```
set.seed(1234)
library(bootstrap) #for the law data
print(cor(law$LSAT, law$GPA)) # A sample of 15 law schools from the population of 82
```

```
## [1] 0.7763745
```

```
print(cor(law82$LSAT, law82$GPA)) # the population correlation
```

```
## [1] 0.7599979
```

Use bootstrap to estimate the standard error of the correlation statistic computed from the sample of scores in `law`.

```r
# Set up the bootstrap
B <- 200  # Number of replicates
n <- nrow(law)  # Sample size
R <- rep(NA, B)  # Storage for replicates

# Bootstrap estimate of standard error of R
for (b in 1:B) {
    # Randomly select the indices
    i <- sample(1:n, size = n, replace = TRUE)
    LSAT <- law$LSAT[i]  # i is a vector of indices
    GPA <- law$GPA[i]
    R[b] <- cor(LSAT, GPA)
}

# Output
print(se.R <- sd(R))
```

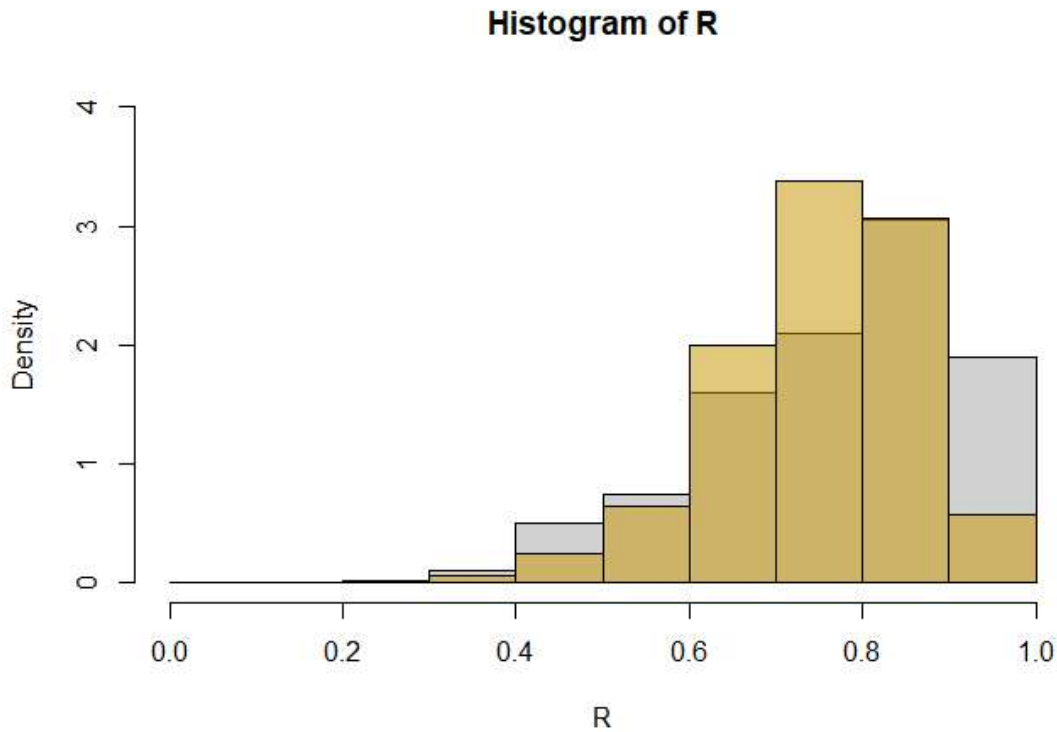
```
## [1] 0.1361966
```

```r
# True standard error of R
R_pop <- rep(NA, 1000)
n_pop <- nrow(law82)
for (j in 1:1000){
    sample <- sample(1:n_pop, size = n)
    LSAT <- law82$LSAT[sample]  # sample is a vector of indices
    GPA <- law82$GPA[sample]
    R_pop[j] <- cor(LSAT, GPA)
}

print(se.Rpop <- sd(R_pop))
```

```
## [1] 0.1124431
```

```r
hist(R, prob = TRUE, breaks = seq(0,1,0.1), ylim = c(0,4))
hist(R_pop, prob = TRUE, add = TRUE, col = rgb(0.8,0.6,0,0.5), breaks = seq(0,1,0.1)
```



The bootstrap estimate of $se(R)$ is 0.136, which is slightly larger than the estimate using the CDF distribution F.

`boot` function

Use `boot` function in the `boot` package to bootstrap.

- Write a function that returns $\hat{\theta}^{(b)}$, where the first argument to the function is the sample data, and the second argument is the vector $\{i_1, \ldots, i_n\}$ of indices.

```
library(boot) #for boot function
r <- function(x, i) {
    # want correlation of columns 1 and 2
    cor(x[i,1], x[i,2])
}

obj <- boot(data = law, statistic = r, R = 2000)
obj
```

```
##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = law, statistic = r, R = 2000)
##
```

```
##
## Bootstrap Statistics :
##      original       bias    std. error
## t1* 0.7763745 -0.006248711   0.1313143
```

```
y <- obj$t
sd(y)
```

```
## [1] 0.1313143
```

The observed value $\hat{\theta}^{(b)}$ of the correlation statistic is labeled `t1*`. The bootstrap estimate of standard error of the estimate is $\hat{se}(\hat{\theta}^{(b)}) = 0.13$, based on 2000 replicates.

Bootstrap estimation of a bias

As the sample mean of the replicates $\{\hat{\theta}^{(b)}\}$ is unbiased for its expected value $E[\hat{\theta}^*]$, the bootstrap estimate of bias is

$$\widehat{\text{bias}}(\hat{\theta}) = \overline{\hat{\theta}}^* - \hat{\theta},$$

where

- $\overline{\hat{\theta}}^* = \frac{1}{B}\sum_{b=1}^{B}\hat{\theta}^{(b)}$;
- $\hat{\theta} = \hat{\theta}(x)$ is the estimate computed from the original observed sample;
- In bootstrap, F_n is sampled from instead of F_X, so we replace θ with $\hat{\theta}$ to estimate the bias;
- Positive bias indicates that $\hat{\theta}$ on average tends to overestimate θ.

Example

Compute the bootstrap estimate of bias in the sample correlation using the `law` dataset.

```
# Sample estimate for n=15
theta.hat <- cor(law$LSAT, law$GPA)

# Bootstrap estimate of bias
B <- 2000 #larger for estimating bias
n <- nrow(law)
theta.b <- numeric(B)
for (b in 1:B) {
  i <- sample(1:n, size = n, replace = TRUE)
  LSAT <- law$LSAT[i]
  GPA <- law$GPA[i]
```

```
  theta.b[b] <- cor(LSAT, GPA)
}
bias <- mean(theta.b - theta.hat)
bias
```

```
## [1] -0.005985165
```

Part VI: *Bootstrap Confidence Intervals*

Standard Normal Bootstrap Confidence Interval

If the sample size is large, then the Central Limit Theorem implies that

$$Z = \frac{\hat{\theta} - E[\hat{\theta}]}{se(\hat{\theta})} \sim N(0,1)$$

where $se(\hat{\theta})$ is the standard error of the estimator.

Hence, if $\hat{\theta}$ is unbiased for θ, then an approximate $100(1-\alpha)\%$ confidence interval for θ is the Z-interval

$$\hat{\theta} \pm z_{\alpha/2} se(\hat{\theta}),$$

where $z_{\alpha/2} = \Phi^{-1}(1-\alpha/2)$.

The standard normal bootstrap confidence interval for θ is a plug-in estimator by replacing $se(\hat{\theta})$ by its bootstrap estimate $\hat{se}^*(\hat{\theta})$:

$$\hat{\theta} \pm z_{\alpha/2} \cdot \hat{se}^*(\hat{\theta}),$$

Assumptions required

- The distribution of $\hat{\theta}$ is normal or $\hat{\theta}$ is a sample mean and the sample size is large
- $\hat{\theta}$ is unbiased for θ

```
theta_hat <- obj$t0
boot_sd <- sd(obj$t)

alpha <- c(.025, .975)
#normal
print(theta_hat + qnorm(alpha) * boot_sd)
```

```
## [1] 0.5190031 1.0337459
```

The standard normal bootstrap confidence interval is the simplest approach, but not necessarily the best.

Basic Bootstrap Confidence Interval

The basic bootstrap confidence interval transforms the distribution of the replicates by subtracting the observed statistic. The quantiles of the transformed sample $\hat{\theta}^* - \hat{\theta}$ are used to determine the confidence limits.

The $100(1-\alpha)\%$ confidence limits for the basic bootstrap confidence interval are

$$(2\hat{\theta} - \hat{\theta}^*_{1-\alpha/2}, \quad 2\hat{\theta} - \hat{\theta}^*_{\alpha/2}),$$

where $\hat{\theta}^*_\alpha$ denotes the α sample quantile of the bootstrap replicates $\hat{\theta}^*$.

Derivation of the Confidence Limits

For a symmetric $100(1-\alpha)\%$ confidence interval (L, U) for θ, we require that

$$\Pr(L \geq \theta) = \Pr(U \leq \theta) = \alpha/2.$$

Then

$$\begin{aligned} \alpha/2 = \Pr(L \geq \theta) &= \Pr(L - \hat{\theta} \geq \theta - \hat{\theta}) \\ &= \Pr(\hat{\theta} - \theta \leq \hat{\theta} - L). \end{aligned}$$

- $L - \hat{\theta}$ should be the $1 - \alpha/2$ percentile of $\hat{\theta} - \theta$, which is unknown
- Based on the bootstrap replicates $\hat{\theta}^*$, $\hat{\theta}^*_{1-\alpha/2} - \hat{\theta}$ is approximately equal to the $1 - \alpha/2$ quantile of $\hat{\theta} - \theta$
- Thus, set $\hat{\theta} - L = \hat{\theta}^*_{1-\alpha/2} - \hat{\theta}$
- Similarly, set $\hat{\theta} - U = \hat{\theta}^*_{\alpha/2} - \hat{\theta}$

Assumptions required

- The distribution is symmetric

```
theta_hat <- obj$t0
alpha <- c(.025, .975)

# basic
boot_quantile <- quantile(obj$t, rev(alpha))
print(2*theta_hat - boot_quantile)
```

```
##     97.5%      2.5%
## 0.5929236 1.0861818
```

Percentile Bootstrap Confidence Interval

- Approximate the quantiles of the sampling distribution of $\hat{\theta}$ by the quantiles of bootstrap samples
- From the ecdf of the replicates, compute the $\alpha/2$ quantile $\hat{\theta}_{\alpha/2}$, and the $1-\alpha/2$ quantile $\hat{\theta}_{1-\alpha/2}$

```
#percentile
print(quantile(obj$t, alpha))
```

```
##      2.5%     97.5%
## 0.4665672 0.9598254
```

`boot.ci` function

The `boot.ci` function allows to compute different types of bootstrap confidence intervals: basic, normal, and percentile.

1. Call `boot` for the bootstrap,
2. Pass the returned `boot` object to `boot.ci` (along with the type you want).

```
boot.obj <- boot(law, R = 2000, statistic = function(x, i){cor(x[i,1], x[i,2])})
print(boot.ci(boot.obj, type=c("basic","norm","perc")))
```

```
## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
## Based on 2000 bootstrap replicates
##
## CALL :
## boot.ci(boot.out = boot.obj, type = c("basic", "norm", "perc"))
##
## Intervals :
## Level      Normal              Basic              Percentile
## 95%   ( 0.5174,  1.0504 )   ( 0.5934,  1.0956 )   ( 0.4571,  0.9593 )
## Calculations and Intervals on Original Scale
```

All three intervals cover the correlation $\rho = 0.76$ of the universe of all law schools in `law82`. One reason for the difference in the percentile and normal confidence intervals could be that the sampling distribution of correlation statistic is not close to normal. When the sampling distribution of the statistic is approximately normal, the percentile interval will agree with the normal interval.

Summary

- Running simulations is an integral part of being a statistician in the 21st century
- R provides us with built-in functions for simulations from a wide variety of distributions
- To make your simulation results reproducible, you must set the seed, using `set.seed()`
- Inverse method generates random numbers based on a sequence of uniform random numbers
- There is a natural connection between iteration, functions, and simulations
- Saving and loading results can be done in two formats: rds and rdata formats
- Bootstrap: make inference on the parameter $\theta = t(F)$ without requiring full knowledge of the true population distribution F
- Step: resample from the sample $(x_1, \ldots, x_n)$ with replacement
- Bootstrap approximations:

$$F \to x_i, i = 1, 2, \ldots, n \to F_n$$

$$F_n \to x^{*(b)}, b = 1, 2, \ldots, B \to F_B^*.$$

- Bootstrap estimation of a standard error

$$\widehat{se}\left(\hat{\theta}^*\right) = \sqrt{\frac{1}{B-1}\sum_{b=1}^{B}\left(\hat{\theta}^{(b)} - \overline{\hat{\theta}^*}\right)^2},$$